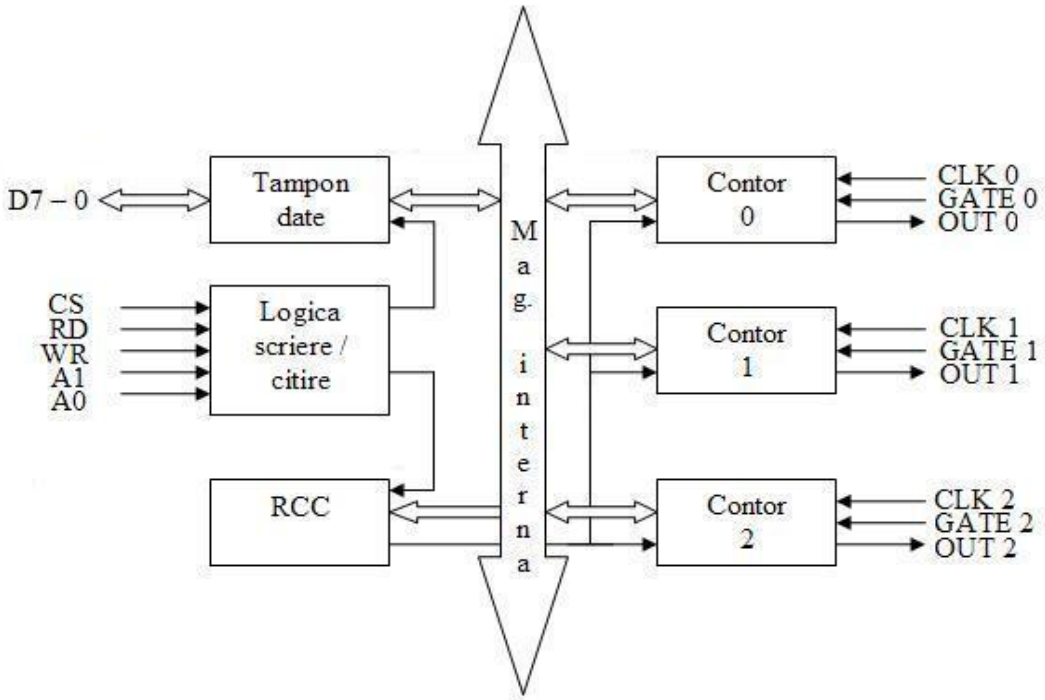
31. Ce este circuitul 8253? Prezentați caracteristici ale sale.

Circuitul 8253 este un generator programabil de impulsuri utilizat în sistemele de calcul pentru cronometrare și numărare. Acesta dispune de trei canale independente, fiecare având un contor pe 16 biți. Circuitul oferă mai multe moduri de operare, inclusiv modul 0, care permite numărarea cu oprire, generând o întrerupere atunci când contorul ajunge la valoarea zero. Modul 1 este folosit pentru generarea de impulsuri retriggerabile, iar modul 2 pentru generarea de unde pătrate cu frecvență variabilă. Modul 3 permite generarea de unde pătrate simetrice, în timp ce modul 4 este utilizat pentru numărarea cu activare unică. Modul 5 permite numărarea cu activare prin hardware, folosind un semnal extern.

Circuitul 8253 poate opera la frecvențe mari, de până la câteva megaherți, în funcție de implementare. Interfața sa paralelă pe 8 biți este compatibilă cu procesoarele din familia x86. Registrele sale de control permit programarea funcțiilor sale, iar circuitul poate fi utilizat și ca temporizator sau divizor de frecvență, esențial pentru generarea semnalelor de ceas.

În trecut, circuitul 8253 a fost utilizat pe scară largă în computerele IBM PC și în aplicațiile embedded, gestionând temporizările pentru diferite funcții ale sistemului. Deși a fost înlocuit în majoritatea cazurilor de soluții mai moderne, circuitul 8253 rămâne un element important în evoluția tehnologică a sistemelor de calcul.

32. Prezentați și comentați structura internă a circuitului 8253.



- A1 şi A0 sunt intrări pentru selectarea unuia din cele 4 porturi interne ale circuitului: 00 pentru Contor 0, 01 pentru Contor 1, 10 pentru Contor 2 şi 11 pentru RCC; se leagă linii din mag. de adrese (de obicei A2 şi A1);

- Tamponul date asigură legătura între circuit şi magistrala de date a sistemului; transferă cuvinte de comandă, constante de divizare precum şi conţinutul contoarelor, în cazul unei operaţii de citire a lor;

- Logica scriere/ citire conduce întregul transfer de date din interiorul circuitului; are ca intrări semnale de comandă din partea microprocesorului care îi comunică ce fel de operaţii se desfăşoară;

- RCC, Registrul de comandă şi control, va prelua cuvintele de comandă transmise de către microprocesor care arată contoarele implicate în operaţiile curente şi modurile de lucru ale acestora; nu poate fi citit;

- Contoarele 0, 1 şi 2 sunt identice şi independente în funcţionare; fiecare este un numărător pe 16 ranguri, cu preâncărcare, care numără în jos, având ca intrare de tact linia CLKi; în faza de

programare, contorul este preâncărcat cu o constantă iar cuvântul de comandă care îi corespunde îi va fixa modul de lucru; fiecare contor poate lucra în unul din 6 moduri; dacă se numără evenimente, impulsurile care le descriu vor fi preluate de contor la intrarea CLK proprie iar dacă se dorește generarea de întârzieri atunci la intrarea CLK a unui contor se va lega tactul sistemului; conţinutul contoarelor poate fi citit de microprocesor; conţinutul contoarelor poate fi citit.

33. Cum se programează circuitul 8253 (nu este necesară structura cuvintelor ci doar ce anume i se comunică circuitului)?

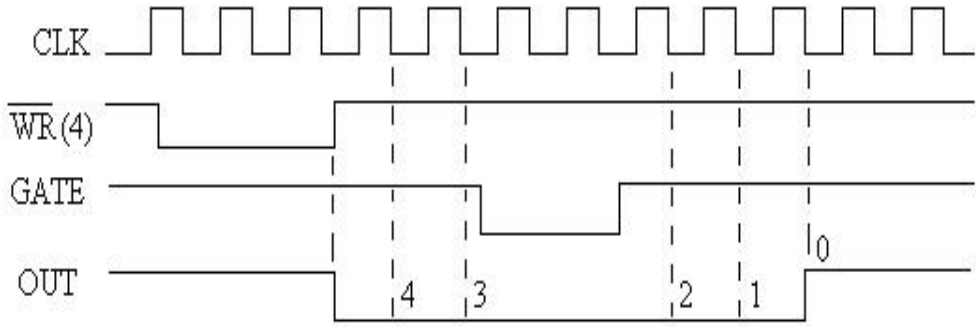
Fiecare contor al circuitului va fi programat individual, fiind privit, de către microprocesor, ca un port de intrare/ ieşire; ordinea poate fi oricare;

- Fiecare contor va fi programat cu un cuvânt de comandă şi cu o constantă de preîncărcare, pe 1 sau 2 octeţi.

34. Descrieți un mod de lucru al unui contor al circuitului 8253.

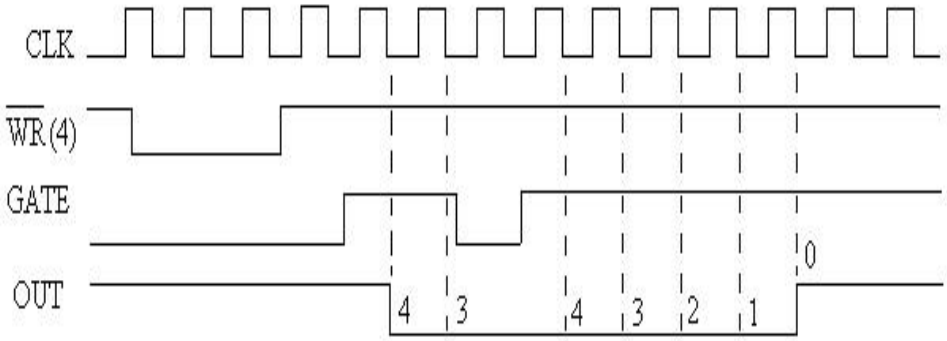
Modul 0 sau generarea unei cereri de întrerupere la sfârşitul numărării:

a. După programarea contorului, ieşirea acestuia trece la nivel 0 logic;
b. Se menţine la această valoare până când conţinutul contorului va ajunge la valoarea 0, în urma numărării impulsurilor sosite pe linia CLK, când va trece la 1 logic;
c. Rămâne în această stare până la o nouă încărcare;
d. Numărarea poate fi inhibată prin activarea intrării GATE;
e. Fig. prezintă modificarea ieşirii unui contor programat să lucreze în modul 0:
    i. Cifra din paranteză indică valoarea constantei;
    ii. Decrementarea începe după încărcarea constantei şi după trecerea unui impuls la intrarea de tact.



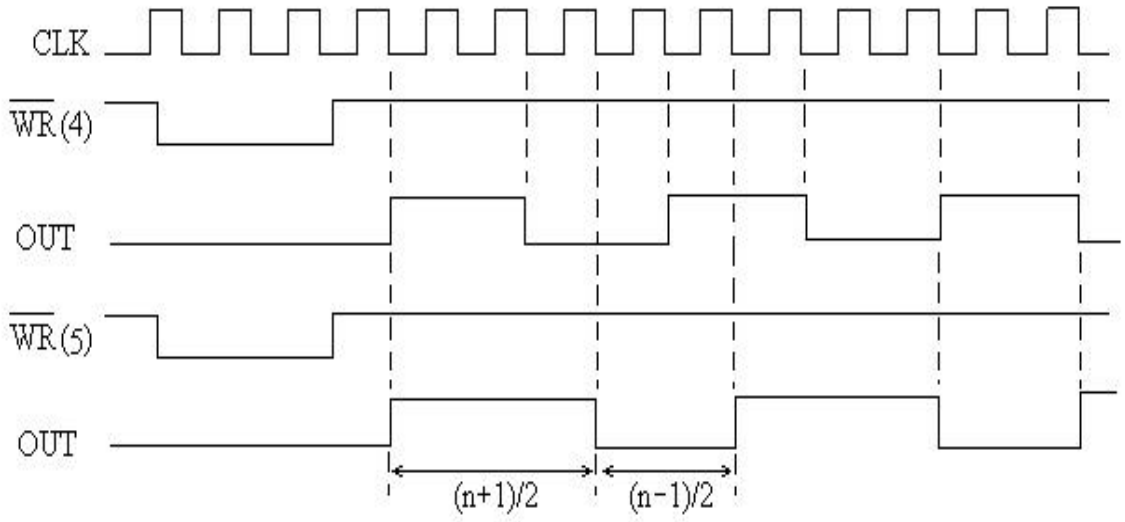
Modul 1 sau monostabil programabil:

- Ieşirea trece în starea 0 logic:
  - după ce are loc încărcarea contorului,
  - după ce are loc o tranziţie pozitivă pe linia GATE şi
  - după ce trece un impuls pe linia CLK.
- Ieşirea rămâne la 0 logic până când contorul ajunge la valoarea 0 după care aceasta revine la 1 logic;
- Dacă, în timpul numărării, este încărcată în contor o nouă valoare aceasta nu va fi luată în considerare dacă nu există tranziţie pozitivă pe linia GATE; o astfel de tranziţie va relua procesul de decrementare de la valoarea iniţială chiar dacă numărarea nu s – a încheiat, iar dacă înaintea tranziţiei contorul a fost înscris cu o nouă constantă, decrementarea va avea loc în conformitate cu noua constantă.
- Modul 1 se mai numeşte şi monostabil programabil întrucât numărarea este anclanşată de o tranziţie pe un semnal întocmai ca la un monostabil la care anclanşarea este comandată tot de o tranziţie pe un semnal.
- Fig. prezintă variaţia ieşirii unui contor în modul 1.



Modul 3 sau generator de semnale dreptunghiulare:

- Ieşirea contorului va genera semnal dreptunghiular cu perioada egală cu perioada tactului înmulţită cu valoarea constantei;
- Factorul de umplere al semnalului va fi 1/ 2 dacă constanta este pară iar dacă aceasta este impară, semnalul va avea 1 logic o durată egală cu (n + 1)T/ 2 şi 0 logic o durată (n – 1)T/ 2, unde n este valoarea constantei iar T este perioada tactului;
- Fig. prezintă variaţia ieşirii unui contor în modul 3:



35. Câte linii sunt necesare pentru un transfer paralel? Justificați.

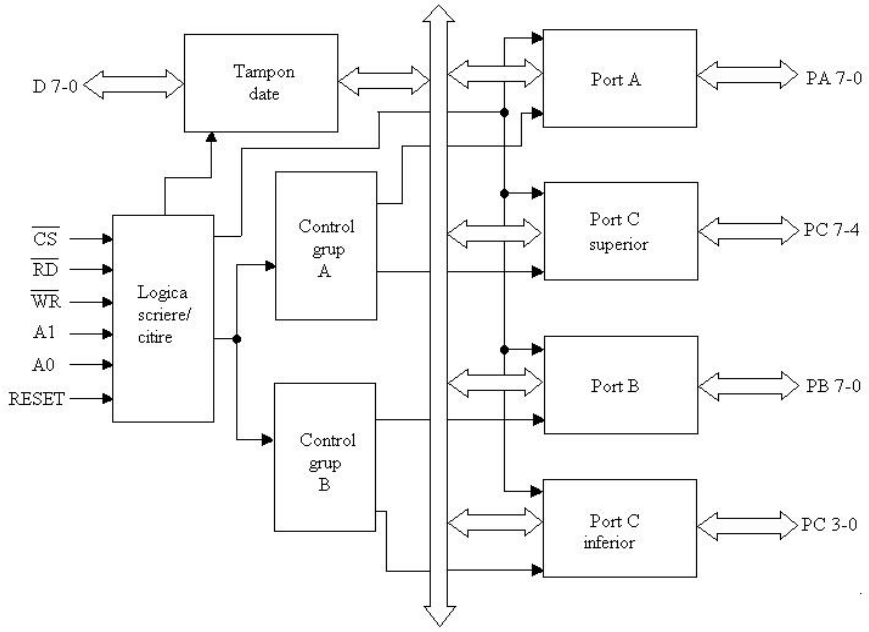
Întrucât se trimite un octet de date, avem nevoie de 8 linii. Mai avem nevoie de cel puţin 2 semnale de dialog – unul din partea transmiţătorului şi unul din partea receptorului, deci încă 2 linii. Mai avem nevoie de încă o linie pentru masa. Prin urmare, pentru un transfer parallel sunt necesare minim 11 linii.

36. Ce este circuitul 8255? Prezentați caracteristici ale sale.

- Dispune de 24 linii de intrare/ ieşire care pot fi configurate în mai multe feluri în funcţie de modul de lucru ales:
  - 2 grupe de câte 12 linii de intrare sau ieşire, fără semnale de dialog;
  - 2 grupe de câte 8 linii de intrare sau ieşire, cu semnale de dialog sau
  - o grupă de 8 linii bidirecţionale, cu semnale de dialog.

- Din cele 24 linii de intrare/ ieşire 16 au posibilităţi de memorare iar 8 nu au această facilitate.
- Comunicarea cu circuitul 8255 se face prin intermediul a 4 adrese de port, corespunzătoare porturilor A, B, C şi portului pentru cuvântul de comandă.

37. Prezentați și comentați structura internă a circuitului 8255.



- A1 şi A0 sunt intrări pentru selectarea unuia din cele 4 porturi interne ale circuitului: 00 pentru Port A, 01 pentru Port B, 10 pentru Port C şi 11 pentru RCC; se leagă linii din mag. de adrese (de obicei A2 şi A1);
- Tamponul date asigură legătura între circuit şi magistrala de date a sistemului: transferă cuvinte de comandă, de stare şi date.
- Logica scriere/ citire are rolul să conducă întregul transfer de date din interiorul circuitului:
  - Are ca intrări semnale de comandă din partea microprocesorului care îi comunică ce fel de operaţii se desfăşoară;
  - Acest bloc comandă cele 2 jumătăţi ale circuitului: grupul A şi grupul B;

- Blocurile de control pentru grupele A şi B primesc comenzi de la blocul Logică scriere/ citire, primesc cuvintele de comandă şi generează comenzile pentru porturile corespunzătoare;

- Porturile A, B şi C au câte 8 biţi:
  - Portul A poate fi ieşire, în modurile 0, 1 sau 2, cu posibilităţi de memorare sau intrare, în modurile 0, 1 sau 2, cu posibilităţi de memorare doar în modul 1 sau 2;
  - Portul B poate fi ieşire, în modurile 0 sau 1, cu posibilităţi de memorare sau intrare, în modurile 0 sau 1, cu posibilităţi de memorare doar în modul 1;
  - Portul C are funcţiuni multiple: poate fi ieşire, doar în modul 0, cu posibilităţi de memorare sau intrare, doar în modul 0, fără posibilităţi de memorare; dacă porturile A şi B lucrează în modurile 1 sau 2, atunci ranguri din portul C vor fi folosite pentru semnale de dialog şi întregul port C poate fi folosit pentru a obţine informaţie de stare de la circuit.

38. Cum se programează circuitul 8255 (nu este necesară structura cuvintelor ci doar ce anume i se comunică circuitului)?

Programarea circuitului:

- Se realizează prin transmiterea unui cuvânt de comandă la adresa portului cuvântului de comandă; se poate cere porturilor să lucreze în unul din următoarele moduri:
  - modul 0, numit şi mod intrare/ ieşire, pentru porturile A, B şi C,
  - modul 1, numit şi mod intrare/ ieşire cu dialog, pentru porturile A şi B şi
  - modul 2, numit şi mod bidirecţional cu dialog, doar pentru portul A.
- Există şi posibilitatea ca, printr – o singură instrucţiune de ieşire, microprocesorul să comande ieşiri individuale ale portului C; se face printr – un cuvânt de comandă în care rangul 7 este la valoarea 0 logic; particularitatea este cunoscută sub numele mod “bit set/ reset” şi este utilă în aplicaţii de control.
- O altă facilitate: generarea unui semnal care poate fi folosit ca cerere de întrerupere; va fi generat în anumite condiţii, care vor fi prezentate, şi, în plus, trebuie validat sistemul de întreruperi al

circuitului; constă în câte un bistabil, pentru modurile de lucru 1 şi 2, iar activarea sau inactivarea bistabilului se face prin ranguri din portul C.

39. Descrieţi unul din modurile de lucru cu dialog ale circuitului 8255.

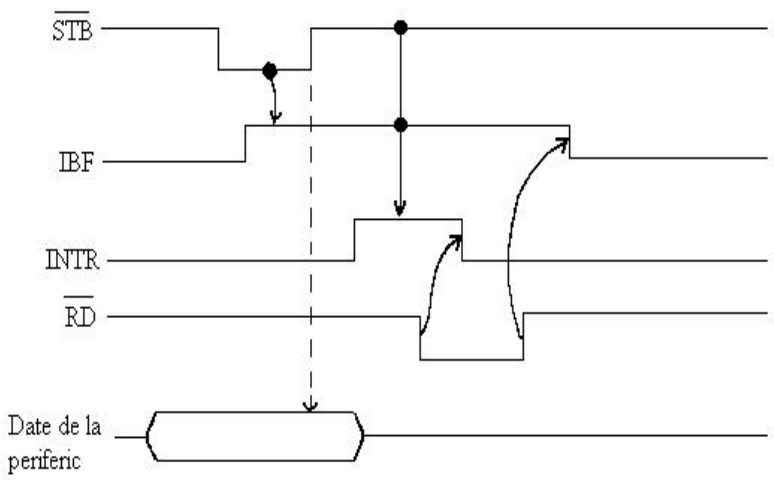
Modul 1(se mai numeşte şi mod de intrare/ ieşire cu dialog şi se foloseşte pentru operaţii de intrare/ ieşire cu dialog). Caracteristicile modului sunt următoarele:

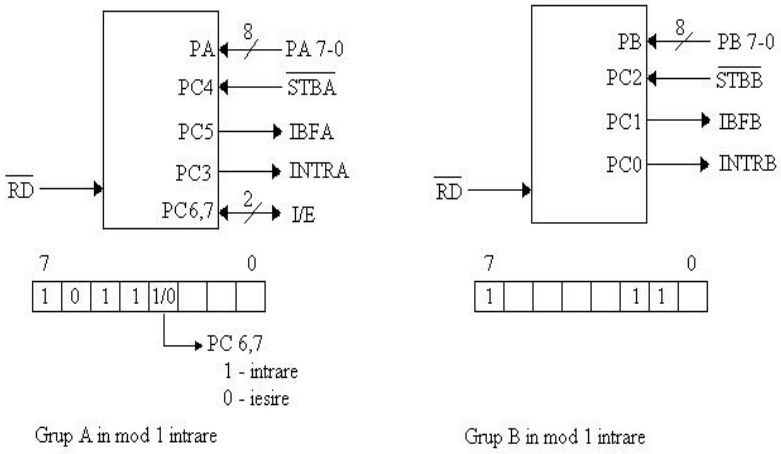
- se pot comanda independent grupul A şi grupul B;
- grupul A conţine portul A şi 3 ranguri ale portului C iar grupul B conţine portul B şi alte 3 ranguri ale portului C;
- rangurile nefolosite ale portului C pot fi comandate prin modul bit set/ reset;
- porturile A şi B pot fi declarate ca intrări sau ieşiri având şi posibilităţi de memorare;

portul C are şi rol de registru de stare.

- Semnalele de dialog folosite în modul 1 intrare sunt:
  - /STB (“Strobe”): intrare prin care perifericul anunţă că a plasat date pe liniile portului; 0 logic va provoca încărcarea datelor în port;
  - IBF (“Input Buffer Full”): ieşire, activă la 1 logic, prin care portul anunţă perifericul că datele au fost preluate; semnalul redevine inactiv după ce semnalul /RD de la microprocesor devine inactiv, ceea ce corespunde situaţiei în care microprocesorul a preluat data încărcată cu /STB;
  - INTR (“Interrupt”): este o ieşire, activă la 1 logic, către microprocesor, prin care 8255 îi comunică acestuia că un port de intrare are o dată de transmis; semnalul poate fi cerere de întrerupere pentru microprocesor; semnalul va fi activ când /STB = 1, adică data a fost încărcată, IBF = 1, adică i s – a comunicat perifericului că data a fost preluată şi dacă sistemul de întreruperi al circuitului este activ; semnalul va fi dezactivat de frontul anterior al semnalului /RD care corespunde situaţiei în care microprocesorul a luat în considerare cererea; sistemul de întreruperi este implementat cu bistabilul INTEA pentru portul A şi INTEB pentru portul B şi este comandat în modul bit set/ reset

prin rangurile PC4 pentru portul A, respectiv PC2 pentru portul B ale portului C.



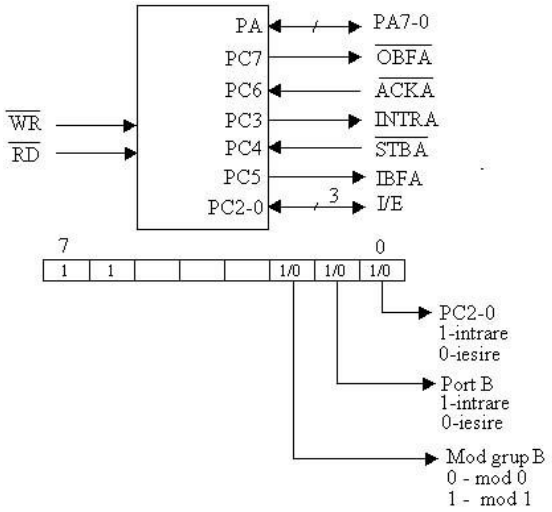


Grup A in mod 1 intrare

Grup B in mod 1 intrare

- Modul 2:
- Se mai numeşte şi mod bidirecţional cu dialog; semnalele de dialog au aceleaşi semnificaţii ca în modul 1, poziţiile lor relative sunt aceleaşi, existând, totuşi, o diferenţă: semnalul /ACK, la 0 logic validează transmiterea datelor de către portul A către periferic iar dacă este 1 logic liniile de date vor fi în a 3 – a stare; /ACK nu mai are semnificaţie de achitare, ci semnificaţie de cerere. Caracteristici:
  - doar grupul A poate lucra în acest mod,
  - grupul A este alcătuit din portul A şi 5 semnale de dialog care folosesc ranguri ale portului C,
  - grupul B poate fi programat în modul 1 sau 0,
  - există facilitatea de memorare şi la intrare şi la ieşire,

- portul C are şi rol de registru de stare,
- Sistemul de întreruperi este implementat de bistabilele INTE1 şi INTE2. INTE1 este asociat ieşirii şi este comandat prin terminalul PC6 iar INTE2 este asociat intrării şi este comandat de terminalul PC4.



40. Care dintre cele 3 posibilități de conectare a porturilor la o UC cu microprocesor este mai rapidă?

Dintre cele trei metode de conectare a porturilor la o unitate centrală (UC) cu microprocesor, **accesul direct la memorie (DMA)** este cea mai rapidă. În acest mod, transferul de date se realizează direct între periferic și memorie, fără a implica procesorul, ceea ce îl face mult mai eficient decât **intrarea/ieșirea programabilă (PIO)**, care necesită intervenția procesorului pentru fiecare transfer, și decât **întreruperile**, care pot limita rata de transfer din cauza întreruperilor frecvente. Astfel, DMA permite o utilizare mai rapidă și mai eficientă a resurselor sistemului.

41. Ce înseamnă DMA și ce resurse cere?

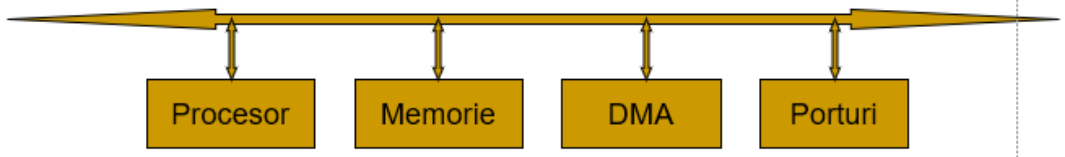
DMA (Direct Memory Access) este o tehnologie care permite transferul direct de date între periferice și memorie, fără a necesita intervenția procesorului. Astfel,

procesorul este eliberat de sarcina de a gestiona aceste transferuri, îmbunătățind performanța sistemului.

Pentru a funcționa, DMA solicită anumite resurse, printre care un **controler DMA**, care coordonează transferul de date între periferice și memorie, și acces direct la **memoria RAM**. De asemenea, pentru transferurile rapide de date, este necesară o **lățime de bandă suficientă a magistralei de date**. În plus, DMA poate folosi **sisteme de întreruperi** pentru a informa procesorul atunci când transferul s-a finalizat sau în caz de erori. Astfel, prin utilizarea acestor resurse, DMA permite transferuri de date eficiente și eliberează procesorul de sarcini suplimentare.

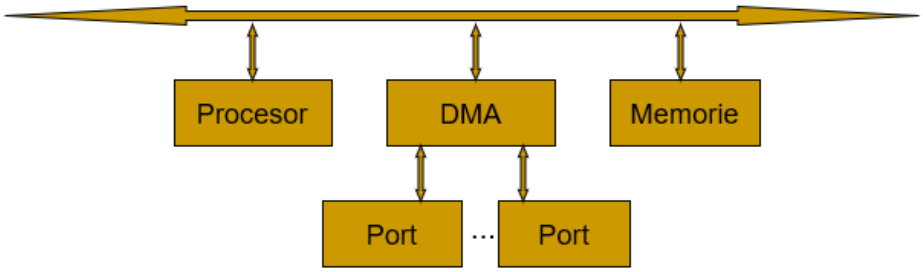
42. Descrieți 3 configurații DMA.

- Magistrală comună, controler DMA separat:



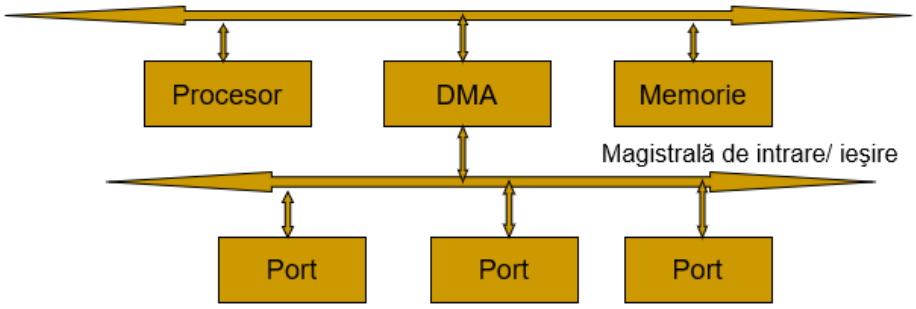
- Fiecare transfer foloseşte magistrala de 2 ori:
  - Port cu DMA,
  - DMA cu memoria
- UC este oprit de 2 ori/ transfer.

- Magistrală comună, controler DMA integrat:



- Controlerul DMA este conectat la mai multe porturi.
- Fiecare transfer foloseşte magistrala o singură dată: DMA cu memoria.
- UC este oprit o singură dată/ transfer.

- Magistrală de intrare/ ieşire distinctă:



- Fiecare transfer foloseşte magistrala comună o singură dată: DMA cu memoria.
- UC este oprit o singură dată/ transfer.

43. Cum realizează un periferic un transfer DMA?

\
Un periferic realizează un transfer DMA prin intermediul unui controler DMA, care gestionează transferul direct de date între periferic și memorie, fără a implica procesorul. Procesul începe cu perifericul care trimite o cerere către controlerul DMA pentru a începe transferul de date. Controlerul DMA este apoi configurat cu

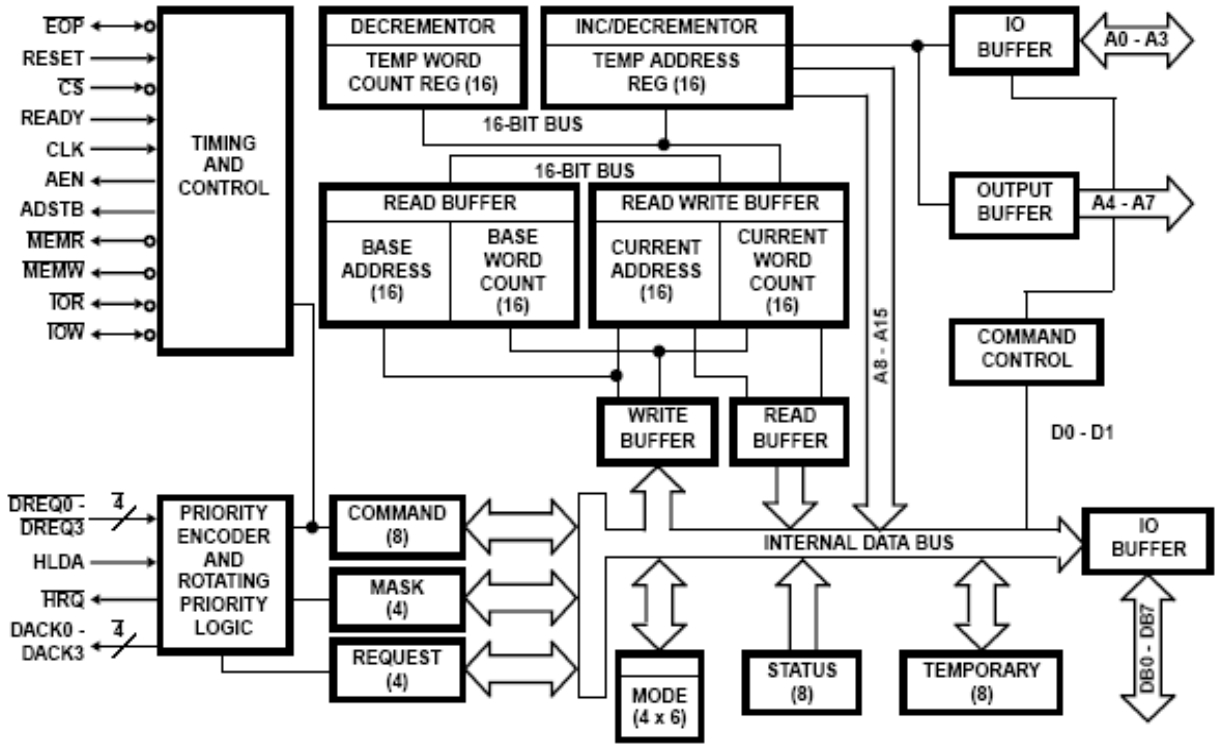
informațiile necesare, precum locația de destinație în memorie și adresa perifericului, și stabilește numărul de date care trebuie transferate.

După configurare, controlerul DMA preia controlul asupra magistralei de date, permițând transferul direct între periferic și memorie, fără intervenția procesorului. Odată ce transferul este complet, controlerul DMA semnalează procesorului că operația s-a încheiat, de obicei printr-o întrerupere. Astfel, transferul DMA permite perifericului să efectueze transferuri de date rapid și eficient, eliberând procesorul de această sarcină.

44. Ce este circuitul 82C37A? Prezentați caracteristicile sale.

Controlerul DMA 82C37A (o variantă mai modernă a clasicului 8237A)

- Dispune de 4 canale independente,
- Poate fi cascadat pentru a obţine oricâte canale,
- Control individual (activare/ dezactivare şi polaritate) asupra fiecărei cereri DREQ şi fiecărui răspuns DACK,
- Permite transfer memorie – memorie,
- Rată de transfer ridicată: până la 4 MO/ sec. sau până la 6.25 MO/ sec. în funcţie de frecvenţa tactului;
- Asigură incrementarea sau decrementarea adresei,
- Se foloseşte cu un registru extern pentru jumătatea mai semnificativă a adresei,
- Poate lucra în mai multe moduri,
- Compatibil TTL/ CMOS. **(CURS VIII pentru moduri de lucru)**



45. Descrieţi întreruperile externe.

Sunt provocate de evenimente externe microprocesorului care cer atenţie imediată din partea acestuia.

- Cel care cere întreruperea trebuie să:
  - Ţină cererea activată până când microprocesorul răspunde;
  - Se identifice atunci când microprocesorul o cere.
- Pot exista cereri simultane din partea mai multor surse => prioritizare:
  - Fie cu o logică de prioritizare:
    - Serială ("daisy – chain") sau
    - Paralelă;
  - Fie cu un PIC;
  - Prioritizarea poate fi:
    - Fixă;
    - Programabilă sau
    - Rotativă.

- Există 2 tipuri de întreruperi externe:

- Mascabile;
- Nemascabile.

- Întreruperile externe mascabile:
  - Vin din partea periferiei sau altui microprocesor (în sisteme multiprocesor);
  - Mascarea se face prin program dezactivând sistemul de întreruperi;
  - Mascarea poate fi totală, acţionând asupra unui indicator din interiorul microprocesorului (soluţie implementată la microprocesoarele Intel şi clone);
  - Mascarea poate fi parţială, permiţând ca doar anumite cereri externe să fie luate în considerare, cele care au un nivel de prioritate mai mare ca unul programat în interiorul microprocesorului (soluţie implementată la unele microprocesoare din familia Freescale);
  - La luarea în considerare a unei cereri de întrerupere mascabile sistemul de întreruperi se dezactivează automat – este un mecanism de protecţie; reactivarea trebuie făcută de programator sau se face implicit în anumite cazuri;

- Întreruperile externe nemascabile:
  - Microprocesorul le ia în seamă întotdeauna;
  - Nu pot fi blocate decât prin hardware extern;
  - Rezervate pentru situaţii catastrofale, de exemplu:
    - O iminentă cădere a tensiunii de alimentare;

O eroare la memorie.

- Microprocesorul le distinge prin:
  - Intrări diferite (în majoritatea cazurilor); de ex. la familia Intel:
    - INTR pentru cereri mascabile şi
    - NMI pentru cereri nemascabile.
  - Cod unic (la unele microprocesoare din familia Freescale).
- Terminale ale microprocesorului implicate:
  - Pentru intrări (mascabile şi nemascabile);
  - Pentru comunicarea acceptării: poate fi:
    - Un terminal dedicat (de ex. INTA la familia Intel) sau
    - O anumită combinaţie pe unele terminale de comandă;

- Magistrala de date (liniile D0 – 7) pentru primirea de către microprocesor a informației de identificare a celui ce a generat cererea).

- Identificarea înseamnă plasarea pe magistrala de date a unui vector de întrerupere în timpul unui ciclu dedicat și poate fi realizată prin:
  - Logică distinctă sau
  - PIC.

- Întreruperile externe sunt de 2 tipuri:
  - Active pe nivel;
  - Active pe front.
- Întreruperile externe active pe nivel:
  - Pe nivel 0 sau 1;
  - Cererea trebuie menţinută activă până când este luată în seamă de microprocesor;
  - Linia trebuie menţinută în starea sa inactivă atunci când nu este activă vreo cerere;
  - Este minimizat riscul luării în seamă a aşa numitelor "Spurious Interrupts".

- Întreruperile externe active pe front:
  - Pe front crescător sau descrescător;
  - Risc de pierdere a unei asemenea cereri dacă nu există vreun mecanism de memorare;
  - Poate fi transformată în o cerere activă pe nivel cu hardware extern (bistabil).

46. Descrieți întreruperile interne.

Se găsesc la microprocesoarele peste 8 biţi şi la toate microcontrolerele;

- Se numesc şi excepţii la unele microprocesoare;
- O parte din ele se numesc şi "traps" la unele microprocesoare;
- Provocate de cauze interne procesorului;
- La microcontrolere pot fi mascate, la microprocesoare nu;
- 2 tipuri de cauze (la microprocesoare):
  - Evenimente interne speciale sau
  - Instrucţiuni dedicate (se mai numesc şi întreruperi software).

- Întreruperi interne provocate de evenimente speciale:
  - Pot fi generate înainte de execuţia unei instrucţiuni: de ex.: o violare de privilegiu sau cod ilegal etc.;
  - Pot fi generate după execuţia unei instrucţiuni: de ex.: o divizare la 0, obţinerea unui rezultat în afara unui domeniu, pas cu pas etc.
- Întreruperi interne datorate instrucţiunilor:
  - Există instrucţiuni ale căror execuţie este echivalentă cu generarea unei cereri de întrerupere; de ex.:
    - INT nn la familia Intel;
    - TRAP #n la microprocesoarele Freescale.
  - Pot fi folosite pentru emularea a noi instrucţiuni.
  - Pe lângă o eventuală prioritizare externă, există şi o prioritizare implicită a surselor de înterupere; de ex. la 8086:
  - Eroare la divizare, INT nn – prioritate maximă;
  - INTO;
  - NMI;
  - INTR;
  - Pas cu pas – prioritate minimă.

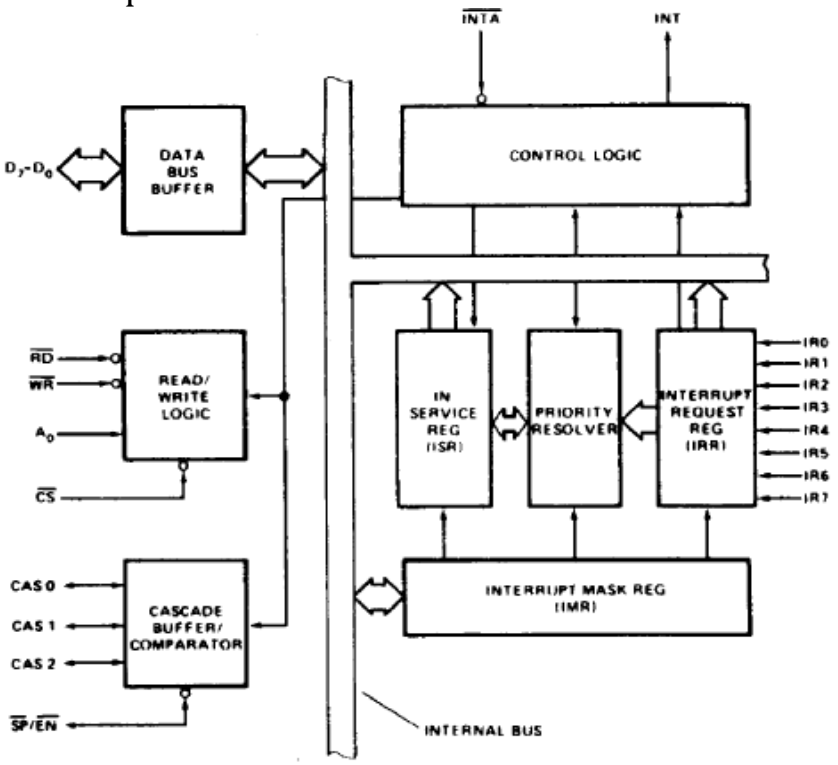
47. Ce este circuitul 8259? Prezentați caracteristicile sale.

Controlerul pentru întreruperi (PIC) 8259A

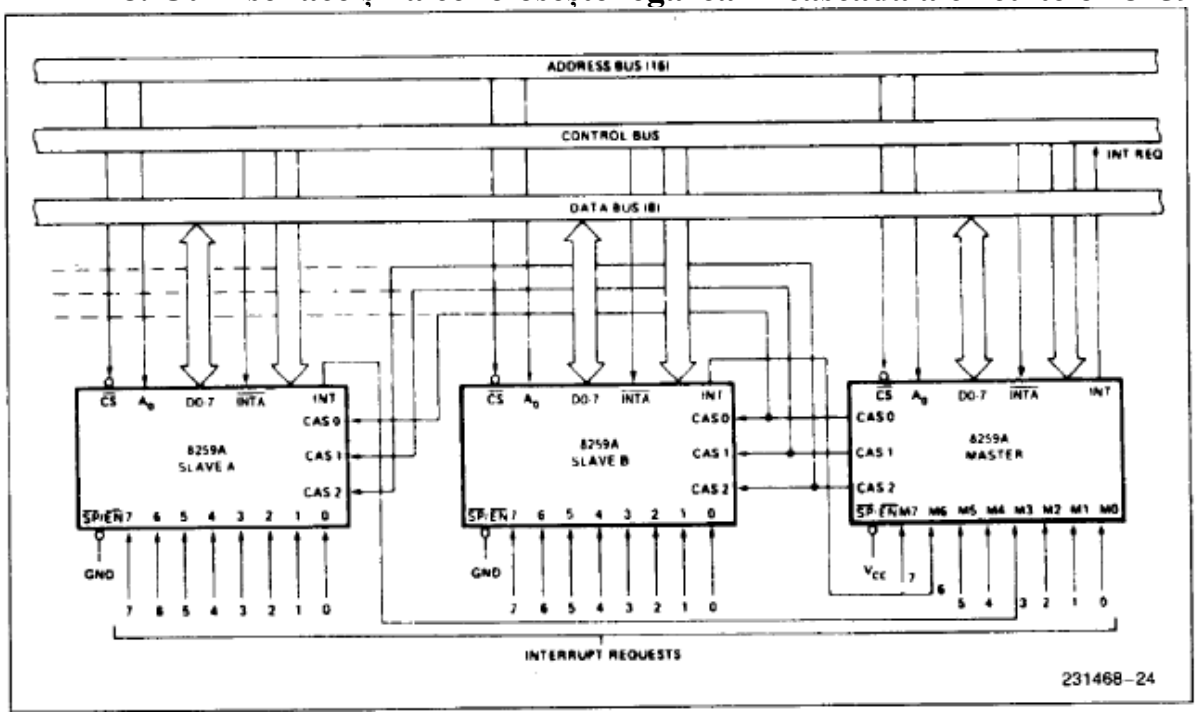
Circuitul respective combină multiple surse de întreruperi ce vin ca input într-o singură întrerupere ce servește drept output pentru microprocesorul gazdă.

- Poate gestiona şi prioritiza 8 cereri de întrerupere;
- Poate fi legat în cascadă cu alte 8 circuite similare asigurând astfel gestionarea şi prioritizarea a 64 cereri de întrerupere;
- Posibilitate de mascare individuală a cererilor;
- Mai multe moduri de gestionare a întreruperilor:
  - Fully nested,
  - Rotating priority,
  - Special mask şi
  - Poll.

- Necesită 2 - 4 cuvinte de inițializare (ICW1 – 4) și 4 cuvinte de operare (OCW1 – 4);
- Generează vectorul de întrerupere asociat cererii luată în considerare de procesor



48. Cum se face și la ce folosește legarea în cascadă a circuitelor 8259?



Structura acestui mecanism este redată de un 8259 ce servește ca master și alte două 8259 ce servesc ca slave-uri, legate prin liniile CAS0, CAS1, CAS2. Sursele de întreruperi se leagă la slave-uri, iar ieșirea ~INT a fiecărui slave nu se duce la procesor, ci se duce la master, iar ~INT-ul de la master se duce la procesor.

O configurație în cascadă ca cea prezentată anterior amplifică numărul de întreruperi care poate fi gestionat de sistem. Cum numărul maxim de interfațare poate fi 9(1 master și 8 slave-uri), rezultă că numărul maxim de întreruperi va fi 64.

49. Justificația existența unui circuit de memorare (bistabil, registru) în schema de comandă a unui led prin program.

Pe magistrala de date circulă în mod continuu informații, adică informația se schimbă rapid pe această magistrală. Ca să vedem un LED luminând, avem nevoie ca acesta să lumineze cel puțin 100ms. O comandă atât de lungă nu poate să fie trimisă pe magistrală de date, adică nu poate să mențină activă o asemenea comandă pentru o perioadă atât de mare. Din acest motiv, avem nevoie de un circuit care memorează, adică un bistabil, care servește ca un port de ieșire.

Secvența ca LED – ul să lumineze este:

```
MOV  AL,00H

OUT  00H,AL
```

50. Este necesar un bistabil pentru citirea stării unui comutator prin program?

Un bistabil nu este necesar pentru citirea stării unui comutator prin program, dar poate fi util în anumite cazuri. De obicei, un microcontroller poate citi direct starea comutatorului printr-un pin digital de intrare. Totuși, un bistabil poate fi folosit pentru a evita fluctuațiile semnalului (debouncing) cauzate de comutatoarele mecanice sau pentru a stoca starea comutatorului între citiri, în aplicații mai complexe sau când este necesară o memorie temporară a stării.

51. Care este diferența între arhitectura și microarhitectura unui microprocesor?

Arhitectura unui microprocesor se referă la designul său la nivel funcțional, adică la setul de instrucțiuni (ISA - Instruction Set Architecture), modul în care procesează datele și interacționează cu memoria și perifericele. Este esențial pentru programatori, deoarece definește cum sunt scrise și executate programele.

Microarhitectura, pe de altă parte, se referă la implementarea fizică și detaliată a arhitecturii, inclusiv cum sunt organizate unitățile interne, cum sunt gestionate registrele, unitățile de execuție și cum sunt optimizate operațiile la nivel hardware. Microarhitectura definește eficiența, performanța și consumul de energie al microprocesorului.

Pe scurt, arhitectura descrie ce face un procesor, iar microarhitectura descrie cum face acest lucru.

52. Prin ce diferă structura Von Neumann de structura Harvard? Ce avantaje și dezavantaje au cele 2 structuri?

Structura Von Neumann și structura Harvard sunt două tipuri de arhitecturi folosite în proiectarea procesorilor, care diferă în modul în care gestionează memoria și fluxul de date.

Structura Von Neumann utilizează o singură memorie comună atât pentru date, cât și pentru instrucțiuni. Aceasta înseamnă că procesorul accesează aceleași linii de memorie pentru a citi atât instrucțiunile, cât și datele, ceea ce poate duce la un conflict între cele două tipuri de acces (această problemă se numește „bottleneck-ul Von Neumann”).

Avantaje:

- Design simplu și costuri de implementare mai mici.
- Memorie unică, ceea ce face ca gestionarea să fie mai simplă.

Dezavantaje:

- Performanță mai scăzută din cauza competiției pentru accesul la memoria comună între instrucțiuni și date.
- Viteză limitată în procesarea paralelă a instrucțiunilor și datelor.

Structura Harvard, pe de altă parte, folosește două memorii separate: una pentru instrucțiuni și alta pentru date. Aceasta permite accesul simultan la ambele tipuri de memorie, ceea ce îmbunătățește performanța, deoarece procesorul poate accesa atât datele cât și instrucțiunile în paralel.

Avantaje:

- Performanță mai mare datorită accesului simultan la memoria de date și instrucțiuni.
- Posibilitatea de a optimiza fiecare tip de memorie pentru scopuri specifice (de exemplu, memorie rapidă pentru instrucțiuni și memorie mai mare pentru date).

Dezavantaje:

- Design mai complex și costuri mai mari de implementare.
- Necesită mai multe linii de comunicație între procesor și memorie.

În concluzie, structura Von Neumann este simplă și mai ieftină, dar are performanțe mai scăzute din cauza limitării la accesul comun la memorie. Structura Harvard oferă o performanță mai bună prin accesul simultan la memorii separate, dar este mai complexă și mai costisitoare.

53. Dați exemple de arhitecturi ale seturilor de instrucțiuni ale microprocesoarelor. Caracteristici.

CISC (Complex Instruction Set Computer)

- ❑ Instrucţiuni complexe, lungi, cu multe moduri de adresare şi cu durată mare de execuţie;
- ❑ Programarea este mai simplă, compilatorul este complex, viteză redusă;
- ❑ Exemple: arhitectura Intel x86, Motorola 68k;

■ RISC (Reduced Instruction Set Computer)

- ❑ Instrucţiuni simple, scurte, cu durată mică de execuţie dar şi cu efect redus;

- ❑ Număr redus de moduri de adresare, instrucțiuni pe 1 sau puține cicluri, compilatoare mai simple, viteză mai mare;
- ❑ Exemple: PowerPC, ARM, SPARC;

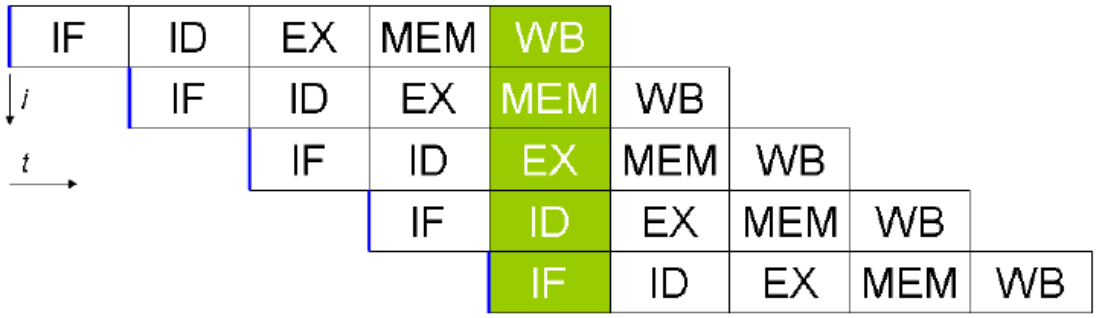
- VLIW (Very Long Instruction Word)
  - ❑ Instrucțiuni lungi, pe mai multe cuvinte;
  - ❑ Dacă procesorul este multicore, se pot grupa mai multe instrucțiuni, 1/core, pentru a rezulta o instrucțiune lungă;

54. Descrieți evoluția soluțiilor implementate în microprocesoare pentru execuția instrucțiunilor (fără hyperthreading).

- Procesoare cu execuție secvențială:

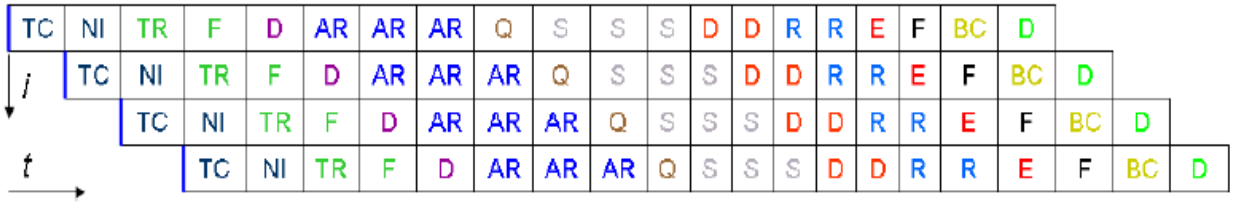
  - ❑ Hardware ineficient folosit, viteză mică;
  - ❑ Simplitate, ușor de stabilit durata secvențelor de cod.

- Procesoare cu bandă de asamblare (pipeline):



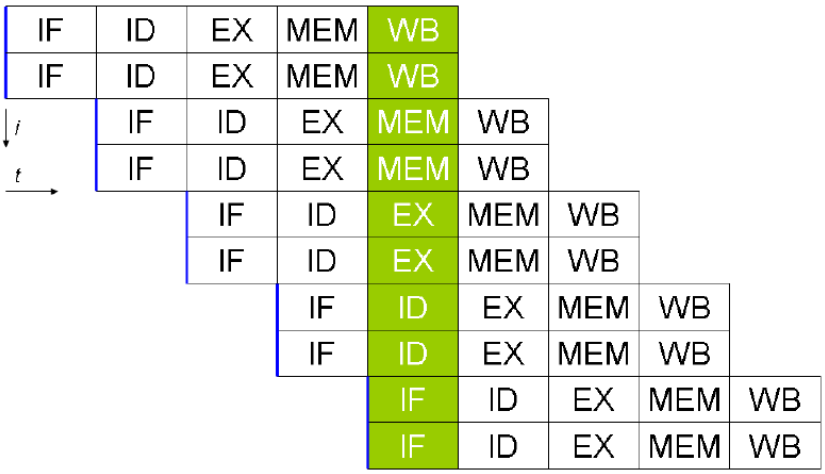
  - ❑ Rata de execuție crește de 5 ori (banda de asamblare are 5 nivele);
  - ❑ Hardware eficient folosit; hardware complex;
  - ❑ Latența este dată de cel mai lent nivel.

- Procesoare cu superpipeline: Pentium 4 cu pipeline cu 20 nivele



- ❑ Nivelele mai lente sunt divizate în nivele cu latență mai mică;
- ❑ Hardware complex.

- Procesoare superscalare:



- ❑ Au un singur nucleu dar mai multe resurse de execuție (multiple ALU, multiple FPU etc.);
- ❑ Execută simultan mai multe instrucțiuni.

- **55. Care este cauza limitării eficienței benzii de asamblare? Prezentați soluții pentru creșterea eficienței benzii de asamblare.**

Creșterea eficienței benzii de asamblare

- ❑ Scade la apariția instrucțiunilor care rup secvențialitatea unui program: salturi, apeluri, întreruperi; pierderea este mare la apariția instrucțiunilor de salt sau apel condiționat;
- ❑ Soluții:
  - Execuția speculativă: se ghicește ramura unui salt/apel condiționat care va fi continuată; în gazul ghicirii eronate, banda trebuie golită; în cazul ghicirii corecte, nu se pierde timp;

- Întârzierea execuţiei: după instrucţiunea de salt/apel condiţionat se plasează o instrucţiune care va fi executată indiferent de ramura care va continua; dacă nu sunt dependenţe de date, această instrucţiune poate fi NOP; se câştigă timp pentru ca instrucţiunea de salt/apel condiţionat să stabilească ramura pe care va continua;
- Predicţia ramurii: se ghicește ramura care se va executa dar nu aleator ci bazat pe anumite considerente, de obicei istoria salturilor;
    - Ex.: execuţia unei bucle; o buclă este executată de mai multe ori înaintea terminării ei, însemnând că au loc mai multe ramificări în o direcţie și doar o singură ramificare în cealaltă direcţie; fie secvenţa în pseudocod HLL.

56. Ce înseamnă superscalaritate?

Superscalaritatea este o arhitectură de procesor care permite executarea simultană a mai multor instrucțiuni într-un singur ciclu de ceas. Aceasta înseamnă că un procesor superscalar poate emite și executa mai multe instrucțiuni în paralel, spre deosebire de un procesor scalar, care execută câte o singură instrucțiune într-un ciclu de ceas.

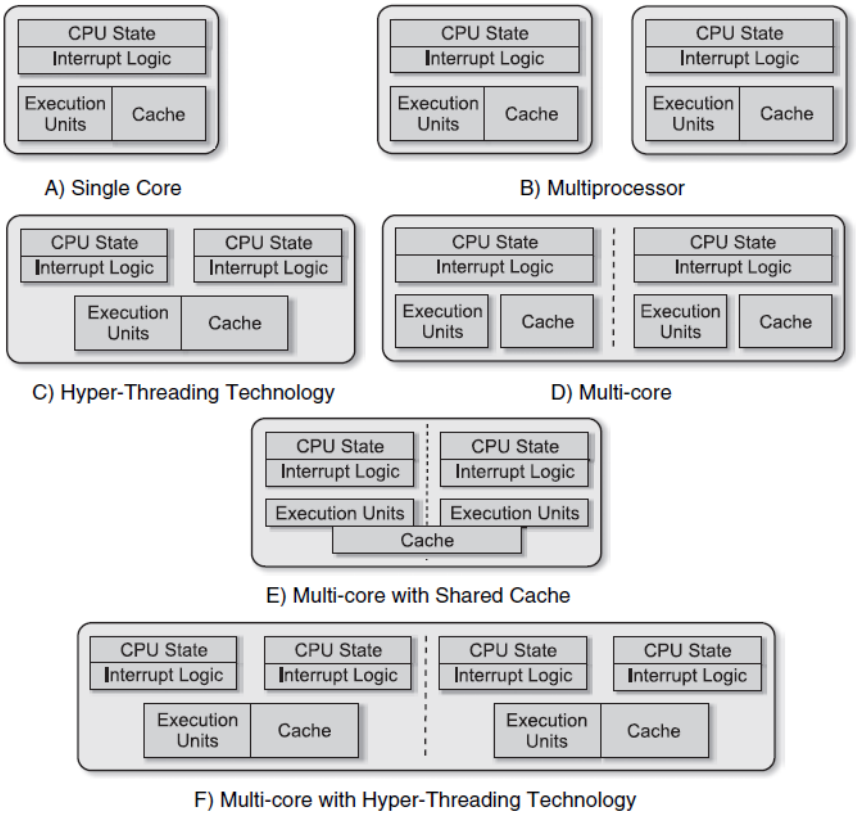
Un procesor superscalar are mai multe unități de execuție (de exemplu, unități aritmetico-logice sau unități de încărcare/stocare) care pot prelua și executa instrucțiuni diferite în același timp, ceea ce îmbunătățește performanța procesorului prin reducerea timpului de execuție pentru programele care pot fi paralelizate.

Superscalaritatea nu înseamnă că toate instrucțiunile pot fi executate simultan, deoarece există limitări legate de dependențele între instrucțiuni (de exemplu, o instrucțiune care depinde de rezultatul altei instrucțiuni) și de capacitatea procesorului de a organiza și gestiona instrucțiunile într-un mod eficient. Totuși, arhitectura superscalară poate duce la o îmbunătățire semnificativă a performanței procesorului, în special în aplicațiile care pot beneficia de paralelism.

57. Care sunt diferențele principiale între microprocesoarele unicore, multimicroprocesoare, microprocesoarele multitasking și cele multicore?

- Microprocesoare unicore
    - Execuţie secvenţială a instrucţiunilor;
    - Facilităţi: pipeline, out-of-order execution (instruction level parallelilsm)
        - Performanţa limitată de interdependenţele între instrucţiuni.
        - Performanţa pipeline-ului limitată de instrucţiunile care modifică secvenţialitatea;
- Multimicroprocesoare: sisteme cu mai multe microprocesoare + circuite, cu acces comun la resurse;
    - Problemă: accesul la resursele comune.
- Microprocesoare multitasking (multithreading)
    - Un unic microprocesor fizic este văzut de software ca mai multe microprocesoare logice;
    - Fiecare microprocesor logic execută un task;
    - Problemă: comutarea taskurilor.
- Microprocesoare multicore
    - Mai multe microprocesoare fizice (core (nucleu)) execută fiecare câte un task.

58. Prezentați schema bloc a unui microprocesor multicore cu hyperthreading.



59. Ce arată legea lui Amdahl?

G. Amdahl a considerat că orice program paralel are o componentă secvențială care nu poate fi paralelizată,*S* și o parte complet paralelizabilă, *P*; timpul de execuție al programului pe un procesor unicore va fi *S + P* iar timpul de execuție pe un processor cu n nuclee va fi *S + P/ n*; atunci FA va fi:

$$Speedup\ (n) = (S + P)/(S + P/n) = 1/((1 - P) + P/n) = 1/(S + P/n) \leq 1/S$$

adică creșterea de viteză este limitată de porțiunea din cod neparalelizată, indiferent de numărul de nuclee;

- este mai eficientă creșterea porțiunii din cod paralelizată decât creșterea numărului nucleelor; ex.: cod paralelizat în raport de 30%, rulând pe un procesor cu 2 nuclee; dublând numărul nucleelor scade timpul de rulare de la 85 unități la 77,5 unități; dublând porțiunea din cod paralelizată, scade timpul de rulare de la 85 unități la 70 unități;

60. Prezentați structura unui microprocesor multicore simetric și a unuia asimetric.

Un microprocesor multicore poate fi conceput în două moduri principale: **simetric** și **asimetric**.

Microprocesor multicore simetric (SMP - Symmetric Multiprocessing)

Un microprocesor multicore simetric este caracterizat prin faptul că toate nucleele sunt identice și au aceleași capabilități. Fiecare nucleu poate executa orice tip de instrucțiune, iar toate nuclee au acces la aceleași resurse, cum ar fi memoria. În această arhitectură, nucleele lucrează simultan, iar controlul și gestionarea sarcinilor sunt distribuite între toate nucleele. Astfel, sarcinile pot fi împărțite între procesoare, fiecare procesor lucrând la o parte din sarcină în paralel, ceea ce duce la o performanță mai bună. Nucleele sunt conectate la o memorie comună și sunt interconectate prin diverse mecanisme de comunicare, precum un bus de date sau o rețea de tip NoC (Network on Chip). Exemple de microprocesoare multicore simetrice sunt Intel Core i7 și AMD Ryzen.

Microprocesor multicore asimetric (AMP - Asymmetric Multiprocessing)

În contrast, un microprocesor multicore asimetric include un nucleu principal (master) care controlează întregul procesor și coordonează execuția sarcinilor. Nucleul principal este responsabil de gestionarea proceselor complexe și poate delega sarcini către nuclee secundare (slaves), care sunt specializate în sarcini mai simple. Aceste nuclee secundare nu pot lua decizii independente și depind de nucleul principal pentru a le ghida activitatea. Memoria poate fi partajată între nucleele principale și secundare, dar în unele cazuri poate fi separată. Arhitectura asimetrică este utilizată în sistemele în care este nevoie de un control centralizat mai puternic, iar sarcinile sunt distribuite în mod eficient între nuclee pe baza complexității lor. Exemple de microprocesoare asimetrice sunt procesoarele utilizate în dispozitivele mobile, unde nucleul principal este folosit pentru sarcini complexe, iar nucleele secundare sunt folosite pentru economisirea energiei.

În concluzie, diferența esențială între cele două arhitecturi este legată de organizarea și funcționarea nucleelor: în arhitectura simetrică, toate nucleele sunt egale și independente, iar în arhitectura asimetrică, nucleul principal coordonează activitatea nucleelor secundare, fiecare având roluri diferite în procesare.